

BUSINESS REPORT

-Orders SQL project-

Project Problem Statement:

You are hired by a chain of online retail stores "Reliant retail limited". They provide you with "orders" database and seek answers to the following queries as the results from these queries will help the company in making data-driven decisions that will impact the overall growth of the online retail store.

Introduction

The goal of this project is to analyze this data and address key business queries that will enable Reliant Retail Limited to make informed decisions. By examining sales trends, customer behavior, product performance, and time-based sales fluctuations, the company can develop strategies that improve profitability, optimize inventory management, and enhance customer retention. The insights derived from this analysis will play a pivotal role in shaping the company's future marketing, sales, and operational strategies, contributing to its overall growth in the highly competitive online retail industry.

Questions to be answered:

1. Write a query to display customer full name with their title (mr/ms), both first name and last name are in upper case with customer email id, customer creation date and display customer's category after applying below categorization rules:
 - i. If customer creation date year <2005 then category a
 - ii. If customer creation date year >=2005 and <2011 then category b
 - iii. If customer creation date year >= 2011 then category c

Hint: Use case statement, no permanent change in table required. [note: tables to be used - online_customer table]

Query:

This query effectively segments customers based on their gender and account creation date while also standardizing email formatting.

```
SELECT
    CASE
        WHEN CUSTOMER_GENDER = 'M' THEN CONCAT('MR. ', UPPER(CUSTOMER_FNAME), ' ', UPPER(CUSTOMER_LNAME))
        WHEN CUSTOMER_GENDER = 'F' THEN CONCAT('MS. ', UPPER(CUSTOMER_FNAME), ' ', UPPER(CUSTOMER_LNAME))
    END AS CUSTOMER_FULL_NAME,
    UPPER(CUSTOMER_EMAIL) AS CUSTOMER_EMAIL_ID,
    CUSTOMER_CREATION_DATE,
    CASE
        WHEN YEAR(CUSTOMER_CREATION_DATE) < 2005 THEN 'CATEGORY A'
        WHEN YEAR(CUSTOMER_CREATION_DATE) >= 2005 AND YEAR(CUSTOMER_CREATION_DATE) < 2011 THEN 'CATEGORY B'
        ELSE 'CATEGORY C'
    END AS CUSTOMER_CATEGORY
FROM
    ONLINE_CUSTOMER;
```

Category	Count of CUSTOMER_CATEGORY
CATEGORY A	3
CATEGORY B	17
CATEGORY C	32
Grand Total	52

This table indicates that the majority of customers fall into **Category C**, followed by **Category B**, while **Category A** has the fewest customers.

2. Write a query to display the following information for the products, which have not been sold: product_id, product_desc, product_quantity_avail, product_price, inventory values(product_quantity_avail*product_price), new_price after applying discount as per the below criteria. Sort the output concerning the decreasing value of inventory_value.

- i. If product price > 20,000 then apply 20% discount
- ii. If product price > 10,000 then apply 15% discount
- iii. If product price <= 10,000 then apply 10% discount

Hint: use case statement, no permanent change in table required. [note: tables to be used - product, order_items table]

Query:

```
SELECT
    P.PRODUCT_ID,
    P.PRODUCT_DESC,
    P.PRODUCT_QUANTITY_AVAIL,
    P.PRODUCT_PRICE,
    (P.PRODUCT_QUANTITY_AVAIL * P.PRODUCT_PRICE) AS INVENTORY_VALUE,
    CASE
        WHEN P.PRODUCT_PRICE > 20000 THEN P.PRODUCT_PRICE * 0.80 -- Apply 20% discount
        WHEN P.PRODUCT_PRICE > 10000 THEN P.PRODUCT_PRICE * 0.85 -- Apply 15% discount
        ELSE P.PRODUCT_PRICE * 0.90 -- Apply 10% discount
    END AS NEW_PRICE
FROM
    PRODUCT P
WHERE
    P.PRODUCT_ID NOT IN (SELECT DISTINCT OI.PRODUCT_ID FROM ORDER_ITEMS OI) -- Exclude sold products
ORDER BY
    INVENTORY_VALUE DESC; -- Sort by inventory value in descending order
```

This SQL query retrieves product information from the PRODUCT table, calculating the total inventory value by multiplying the available quantity by the product price. It applies a tiered

discount: 20% for products priced above 20,000, 15% for those between 10,001 and 20,000, and 10% for products priced at 10,000 or below, creating a new discounted price column (NEW_PRICE). The query excludes products that have already been sold by filtering out product IDs found in the ORDER_ITEMS table, and it sorts the results in descending order of inventory value, highlighting unsold products with the highest stock value.

PRODUCT_ID	PRODUCT_DESC	PRODUCT_QUANTITY_AVAILABLE	PRODUCT_PRICE	INVENTORY_VALUE	NEW_PRICE
99999	Samsung Galaxy Tab 2 P3100	50	19300	965000	16405
99997	Sony Xperia U (Black White)	50	16499	824950	14024.15
99998	Nikon Coolpix L810 Bridge	50	14987	749350	12738.95
99995	LG MS-2049UW Solo Microwave	100	4800	480000	4320
99996	Nokia Asha 200 (Graphite)	100	4070	407000	3663
99994	HP Deskjet 2050 All-in-One - J510a Printer	100	3749	374900	3374.1
99991	Dell Targus Synergy 2.0 Backpack	250	999	249750	899.1
99992	Tom Clancy's Ghost Recon: Future Soldier (PC Game)	250	999	249750	899.1
99993	Nokia 1280 (Black)	250	999	249750	899.1
99990	Quanta 4 Port USB Hub	50	500	25000	450
230	Esprit Analog Watch	5	3495	17475	3145.5
249	All-in-one Board Game	20	450	9000	405
250	Huwi Wi-Fi Receiver 500Mbps	30	287	8610	258.3

The highest inventory value belongs to the **Samsung Galaxy Tab 2 P3100** with 50 units, priced at 19,300, resulting in a total inventory value of 965,000, and after applying a 15% discount, its new price is 16,405. Similarly, other products like the **Sony Xperia U**, **Nikon Coolpix L810**, and **HP Deskjet Printer** have their respective discounted prices based on the discount structure, with some products priced lower than 10,000 receiving a 10% discount. The products are ranked by inventory value, showing both high-ticket items like electronics and lower-cost items such as the **Quanta USB Hub** and **All-in-one Board Game**.

3. write a query to display product_class_code, product_class_description, count of product type in each product class, and inventory value (p.product_quantity_avail*p.product_price). Information should be displayed for only those product_class_code that have more than 1,00,000 inventory value. sort the output concerning the decreasing value of inventory_value. [note: tables to be used- product, product_class]

Query:

```
SELECT
    PC.PRODUCT_CLASS_CODE,
    PC.PRODUCT_CLASS_DESC,
    COUNT(PC.PRODUCT_CLASS_CODE) AS COUNT_OF_PRODUCT_TYPE,
    SUM((P.PRODUCT_QUANTITY_AVAIL*P.PRODUCT_PRICE)) AS INVENTORY_VALUE
FROM
    PRODUCT P
INNER JOIN
    PRODUCT_CLASS PC ON P.PRODUCT_CLASS_CODE = PC.PRODUCT_CLASS_CODE
GROUP BY
    PC.PRODUCT_CLASS_CODE, PC.PRODUCT_CLASS_DESC
HAVING
    INVENTORY_VALUE >100000
ORDER BY
    INVENTORY_VALUE DESC;
```

This SQL query retrieves product class details, counting the number of products in each class and calculating the total inventory value by multiplying product quantities by their prices. It filters the results to include only product classes with an inventory value greater than 100,000 and sorts them in descending order of inventory value. The output provides key insights into high-value product classes, helping identify the most valuable product categories based on their stock levels and pricing.

PRODUCT_CLASS_CODE	PRODUCT_CLASS_DESCRIPTION	COUNT_OF_PRODUCT_TYPE	INVENTORY_VALUE
3000	Promotion-High Value	4	2564300
2050	Electronics	4	1665600
3001	Promotion-Medium Value	3	1261900
2055	Mobiles	2	1092500
3002	Promotion-Low Value	3	749250
2052	Clothes	4	410000
2051	Toys	5	194100
2057	Watches	4	178820
2059	Bags	5	115170

The **Promotion-High Value** class holds the highest inventory value at 2,564,300 with 4 product types, followed by the **Electronics** class with 1,665,600 and 4 product types. Other notable classes include **Promotion-Medium Value** and **Mobiles**, with inventory values exceeding 1,000,000. Lower-value classes like **Clothes**, **Toys**, **Watches**, and **Bags** have inventory values ranging from 115,170 to 410,000, with a higher count of product types in categories like **Toys** and **Bags**. The data highlights which product categories contribute most significantly to the company's inventory value.

4. Write a query to display customer_id, full name, customer_email, customer_phone and country of customers who have cancelled all the orders placed by them(use sub-query)

[note: tables to be used - online_customer, addresss, order_header]

Query:

```
SELECT
    OC.CUSTOMER_ID,
    CASE
        WHEN OC.CUSTOMER_GENDER = 'M' THEN CONCAT('MR. ', UPPER(OC.CUSTOMER_FNAME), ' ', UPPER(OC.CUSTOMER_LNAME))
        WHEN OC.CUSTOMER_GENDER = 'F' THEN CONCAT('MS. ', UPPER(OC.CUSTOMER_FNAME), ' ', UPPER(OC.CUSTOMER_LNAME))
    END AS CUSTOMER_FULL_NAME,
    OC.CUSTOMER_EMAIL,
    OC.CUSTOMER_PHONE,
    A.COUNTRY
FROM
    ONLINE_CUSTOMER OC
JOIN
    ADDRESS A ON OC.ADDRESS_ID=A.ADDRESS_ID
WHERE
    OC.CUSTOMER_ID IN (
        SELECT OH.CUSTOMER_ID
        FROM ORDER_HEADER OH
        WHERE OH.ORDER_STATUS='Cancelled'
        GROUP BY OH.CUSTOMER_ID
        HAVING COUNT(*)=(SELECT COUNT(*) FROM ORDER_HEADER WHERE CUSTOMER_ID = oh.CUSTOMER_ID)
    );
```

The query effectively identifies customers who have canceled every order they placed, providing a detailed view of their contact information and location. This information could be useful for customer retention strategies or further analysis of cancellation reasons.

CUSTOMER_ID	CUSTOMER_FULL_NAME	CUSTOMER_EMAIL	CUSTOMER_PHONE	COUNTRY
41	MR. THARMAN SHANMUGARATNAM	tharshan@yahoo.co.s g	8.57E+09	Singapore

This entry indicates that **Mr. Tharman Shanmugaratnam** is a customer based in Singapore, and his contact information is available for follow-up or analysis related to the cancellation of orders. This

information could be beneficial for understanding customer behavior and implementing strategies to address cancellations or enhance customer satisfaction.

5. Write a query to display shipper name, city to which it is catering, number of customer catered by the shipper in the city and number of consignments delivered to that city for shipper dhl

[note: tables to be used -shipper, online_customer, addresss, order_header]

Query:

```
SELECT
    S.SHIPPER_NAME,
    A.CITY AS CATERING_CITY,
    COUNT(DISTINCT OC.CUSTOMER_ID) AS CUSTOMER_CATERED,
    COUNT(*) AS CONSIGNMENTS_DELIVERED
FROM
    SHIPPER S
JOIN
    ORDER_HEADER OH ON S.SHIPPER_ID = OH.SHIPPER_ID
JOIN
    ONLINE_CUSTOMER OC ON OH.CUSTOMER_ID = OC.CUSTOMER_ID
JOIN
    ADDRESS A ON OC.ADDRESS_ID = A.ADDRESS_ID
WHERE
    S.SHIPPER_NAME = 'DHL'
GROUP BY
    S.SHIPPER_NAME, A.CITY;
```

This query retrieves delivery performance metrics for the shipper **DHL**, focusing on the number of unique customers served and the total consignments delivered across different cities. It joins the SHIPPER, ORDER_HEADER, ONLINE_CUSTOMER, and ADDRESS tables to access relevant information about shipments and customer locations. The query filters result to include only those pertaining to DHL, grouping the data by shipper name and city. The output summarizes the number of distinct customers catered to and the total consignments delivered, providing insights into DHL's operational reach and customer engagement in various urban areas.

SHIPPER_NAME	CATERING_CITY	CUSTOMER_CATERED	CONSIGNMENTS_DELIVERED
DHL	Abington	1	1
DHL	Amherst	1	1
DHL	Bangalore	3	5
DHL	Birmingham	1	1
DHL	Brooklyn	1	1
DHL	Dharmapuri	1	1
DHL	Hosur	1	1
DHL	Hyderabad	2	2
DHL	W. Alibio	1	1

The table shows DHL's delivery operations, indicating that they serve a small number of customers in various cities, with Bangalore being a notable hub for their consignments. This information could be useful for assessing regional performance and planning logistics strategies.

6. Write a query to display customer id, customer full name, total quantity and total value (quantity*price) shipped where mode of payment is cash and customer last name starts with 'g'

[note: tables to be used -online_customer, order_items, product, order_header]

Query:

```
SELECT
    OC.CUSTOMER_ID,
    CASE
        WHEN OC.CUSTOMER_GENDER = 'M' THEN CONCAT('MR. ', UPPER(OC.CUSTOMER_FNAME), ' ', UPPER(OC.CUSTOMER_LNAME))
        WHEN OC.CUSTOMER_GENDER = 'F' THEN CONCAT('MS. ', UPPER(OC.CUSTOMER_FNAME), ' ', UPPER(OC.CUSTOMER_LNAME))
    END AS CUSTOMER_FULL_NAME,
    SUM(OI.PRODUCT_QUANTITY) AS TOTAL_QUANTITY,
    SUM(OI.PRODUCT_QUANTITY * P.PRODUCT_PRICE) AS TOTAL_VALUE_SHIPPED
FROM
    ONLINE_CUSTOMER OC
JOIN
    ORDER_HEADER OH ON OC.CUSTOMER_ID = OH.CUSTOMER_ID
JOIN
    ORDER_ITEMS OI ON OH.ORDER_ID = OI.ORDER_ID
JOIN
    PRODUCT P ON OI.PRODUCT_ID = P.PRODUCT_ID
WHERE
    OH.PAYMENT_MODE = 'Cash'
    AND OC.CUSTOMER_LNAME LIKE 'G%'
GROUP BY
    OC.CUSTOMER_ID, OC.CUSTOMER_FNAME, OC.CUSTOMER_LNAME;
```

The query effectively summarizes the order details for customers with last names starting with 'G' who opted for cash payments, providing insights into their purchasing behavior in terms of quantity and monetary value of products shipped.

CUSTOMER_ID	CUSTOMER_FULL_NAME	TOTAL_QUANTITY	TOTAL_VALUE_SHIPPED
6	MS. ANITA GOSWAMI	25	93237
24	MR. BRIAN GRAZER	4	4010

The table highlights the purchasing behaviour of these two customers, showing a significant difference in total quantity ordered and value shipped, with Ms. Goswami being a more substantial customer in terms of both metrics.

7. Write a query to display order_id and volume of biggest order (in terms of volume) that can fit in carton id 10

-- [note: tables to be used -carton, order_items, product]

Query:

```
SELECT
    OI.ORDER_ID,
    MAX(P.LEN * P.WIDTH * P.HEIGHT) AS BIGGEST_VOLUME
FROM
    ORDER_ITEMS OI
JOIN
    PRODUCT P ON OI.PRODUCT_ID = P.PRODUCT_ID
JOIN
    CARTON C ON P.LEN <= C.LEN AND P.WIDTH <= C.WIDTH AND P.HEIGHT <= C.HEIGHT
WHERE
    C.CARTON_ID=10
GROUP BY
    OI.ORDER_ID
ORDER BY
    BIGGEST_VOLUME DESC
LIMIT 1;
```

This query effectively identifies the order that contains the product with the largest volume suitable for carton **ID = 10**. This information can be valuable for logistics and packaging decisions, ensuring that products are appropriately stored and shipped based on their dimensions.

ORDER_ID	BIGGEST_VOLUME
10001	17880000

The output indicates that the order with **ORDER_ID 10001** contains the product with the largest volume, measuring **17,880,000** cubic units. This significant volume suggests that the product is likely quite large or bulky, making it essential for logistical considerations such as packaging, storage, and shipping. Identifying this order can help in optimizing space and resources during the fulfillment process, ensuring that it is managed effectively to accommodate the size of the item.

8. Write a query to display product_id, product_desc, product_quantity_avail, quantity sold, and show inventory status of products as below as per below condition:

- a. For electronics and computer categories,
 - i. If sales till date is zero then show 'no sales in past, give discount to reduce inventory',
 - ii. If inventory quantity is less than 10% of quantity sold, show 'low inventory, need to add inventory',
 - iii. If inventory quantity is less than 50% of quantity sold, show 'medium inventory, need to add some inventory',
 - iv. If inventory quantity is more or equal to 50% of quantity sold, show 'sufficient inventory'
 - b. For mobiles and watches categories,
 - i. If sales till date is zero then show 'no sales in past, give discount to reduce inventory',
 - ii. If inventory quantity is less than 20% of quantity sold, show 'low inventory, need to add inventory',
 - iii. If inventory quantity is less than 60% of quantity sold, show 'medium inventory, need to add some inventory',
 - iv. If inventory quantity is more or equal to 60% of quantity sold, show 'sufficient inventory'
 - c. Rest of the categories,
 - i. If sales till date is zero then show 'no sales in past, give discount to reduce inventory',
 - ii. If inventory quantity is less than 30% of quantity sold, show 'low inventory, need to add inventory',
 - iii. If inventory quantity is less than 70% of quantity sold, show 'medium inventory, need to add some inventory',
 - iv. If inventory quantity is more or equal to 70% of quantity sold, show 'sufficient inventory'
- [note: tables to be used -product, product_class, order_items] (use sub-query)

Query:

```
SELECT
    P.PRODUCT_ID,
    P.PRODUCT_DESC,
    P.PRODUCT_QUANTITY_AVAIL,
    SUM(OI.PRODUCT_QUANTITY) AS QUANTITY_SOLD,
    CASE
        WHEN PC.PRODUCT_CLASS_CODE IN ('Electronics', 'Computer') THEN
            CASE
                WHEN SUM(OI.PRODUCT_QUANTITY) = 0 THEN 'NO SALES IN PAST, GIVE DISCOUNT TO REDUCE INVENTORY'
                WHEN P.PRODUCT_QUANTITY_AVAIL < (0.1 * SUM(OI.PRODUCT_QUANTITY)) THEN 'LOW INVENTORY, NEED TO ADD INVENTORY'
                WHEN P.PRODUCT_QUANTITY_AVAIL < (0.5 * SUM(OI.PRODUCT_QUANTITY)) THEN 'MEDIUM INVENTORY, NEED TO ADD SOME INVENTORY'
                ELSE 'SUFFICIENT INVENTORY'
            END
        WHEN PC.PRODUCT_CLASS_CODE IN ('Mobiles', 'Watches') THEN
            CASE
                WHEN SUM(OI.PRODUCT_QUANTITY) = 0 THEN 'NO SALES IN PAST, GIVE DISCOUNT TO REDUCE INVENTORY'
                WHEN P.PRODUCT_QUANTITY_AVAIL < (0.2 * SUM(OI.PRODUCT_QUANTITY)) THEN 'LOW INVENTORY, NEED TO ADD INVENTORY'
                WHEN P.PRODUCT_QUANTITY_AVAIL < (0.6 * SUM(OI.PRODUCT_QUANTITY)) THEN 'MEDIUM INVENTORY, NEED TO ADD SOME INVENTORY'
                ELSE 'SUFFICIENT INVENTORY'
            END
        ELSE
            CASE
                WHEN SUM(OI.PRODUCT_QUANTITY) = 0 THEN 'NO SALES IN PAST, GIVE DISCOUNT TO REDUCE INVENTORY'
                WHEN P.PRODUCT_QUANTITY_AVAIL < (0.3 * SUM(OI.PRODUCT_QUANTITY)) THEN 'LOW INVENTORY, NEED TO ADD INVENTORY'
                WHEN P.PRODUCT_QUANTITY_AVAIL < (0.7 * SUM(OI.PRODUCT_QUANTITY)) THEN 'MEDIUM INVENTORY, NEED TO ADD SOME INVENTORY'
                ELSE 'SUFFICIENT INVENTORY'
            END
        END AS INVENTORY_STATUS
FROM
    PRODUCT P
JOIN
    PRODUCT_CLASS PC ON P.PRODUCT_CLASS_CODE = PC.PRODUCT_CLASS_CODE
LEFT JOIN
    ORDER_ITEMS OI ON P.PRODUCT_ID = OI.PRODUCT_ID
GROUP BY
    P.PRODUCT_ID, P.PRODUCT_DESC, P.PRODUCT_QUANTITY_AVAIL, PC.PRODUCT_CLASS_CODE
ORDER BY
    P.PRODUCT_ID ASC;
```

The query provides a comprehensive view of product inventory statuses, helping businesses make informed decisions about restocking and promotions. It identifies products that may need discounts to reduce excess inventory or those that require restocking based on their sales performance, facilitating better inventory management.

Row Labels	Count of PRODUCT_DESC
LOW INVENTORY, NEED TO ADD INVENTORY	4
MEDIUM INVENTORY, NEED TO ADD SOME INVENTORY	2
SUFFICIENT INVENTORY	54
Grand Total	60

This inventory analysis reveals that while most products have sufficient stock, there are a small number that requires attention for restocking. The company can focus on these low and medium inventory items to ensure that they meet customer demand effectively, while also avoiding overstocking on items that are selling well.

9. Write a query to display product_id, product_desc and total quantity of products which are sold together with product id 201 and are not shipped to city bangalore and new delhi. Display the output in descending order concerning tot_qty.(use sub-query)
[note: tables to be used -order_items,product,order_header, online_customer, address]

Query:

```
SELECT
    P.PRODUCT_ID,
    P.PRODUCT_DESC,
    SUM(OI.PRODUCT_QUANTITY) AS TOT_QTY
FROM
    ORDER_ITEMS OI
INNER JOIN
    PRODUCT P ON OI.PRODUCT_ID = P.PRODUCT_ID
WHERE
    OI.ORDER_ID IN (
        SELECT
            OI.ORDER_ID
        FROM
            ORDER_ITEMS OI
        JOIN
            ORDER_HEADER OH ON OI.ORDER_ID = OH.ORDER_ID
        JOIN
            ONLINE_CUSTOMER OC ON OH.CUSTOMER_ID = OC.CUSTOMER_ID
        JOIN
            ADDRESS A ON OC.ADDRESS_ID = A.ADDRESS_ID
        WHERE
            OI.PRODUCT_ID = 201
            AND OH.ORDER_STATUS = 'Shipped'
            AND A.CITY NOT IN ('New Delhi', 'Bangalore')
    )
    AND P.PRODUCT_ID != 201
GROUP BY
    P.PRODUCT_ID, P.PRODUCT_DESC
ORDER BY
    TOT_QTY DESC;
```

The query effectively identifies which products have been sold alongside a specific product (with PRODUCT_ID 201) in orders shipped outside of the specified cities, enabling better inventory and sales analysis. This information could be useful for targeted marketing strategies, inventory replenishment decisions, or identifying product performance trends in various regions.

PRODUCT_ID	PRODUCT_DESC	TOT_QTY
216	External Hard Disk 500 GB	3
207	Remote Control Car	2
202	Sams 192 L4 Single-door Refrigerator	1
212	Samsung Galaxy On6	1
214	Harry Potter	1

In total, the query retrieved information about **5** different products sold alongside the specified product in orders shipped to regions outside the excluded cities. The data indicates varying levels of interest among customers for these products, with the external hard disk being the most popular.

10. Write a query to display the order_id, customer_id and customer fullname and total quantity of products shipped for order ids which are even and shipped to address where pincode is not starting with "5"

[note: tables to be used - online_customer, order_header, order_items, address]

Query:

```

SELECT
    OH.ORDER_ID,
    OC.CUSTOMER_ID,
    CASE
        WHEN OC.CUSTOMER_GENDER = 'M' THEN CONCAT('MR. ', UPPER(OC.CUSTOMER_FNAME), ' ', UPPER(OC.CUSTOMER_LNAME))
        WHEN OC.CUSTOMER_GENDER = 'F' THEN CONCAT('MS. ', UPPER(OC.CUSTOMER_FNAME), ' ', UPPER(OC.CUSTOMER_LNAME))
    END AS CUSTOMER_FULL_NAME,
    SUM(OI.PRODUCT_QUANTITY) AS TOT_QTY
FROM
    ORDER_HEADER OH
INNER JOIN
    ONLINE_CUSTOMER OC ON OH.CUSTOMER_ID = OC.CUSTOMER_ID
INNER JOIN
    ADDRESS A ON OC.ADDRESS_ID = A.ADDRESS_ID
INNER JOIN
    ORDER_ITEMS OI ON OH.ORDER_ID = OI.ORDER_ID
WHERE
    OI.ORDER_ID % 2 = 0
    AND OH.ORDER_STATUS = 'Shipped'
    AND A.PINCODE NOT LIKE '5%'
GROUP BY
    OH.ORDER_ID, OC.CUSTOMER_ID, CUSTOMER_FULL_NAME;

```

The output of this query provides insights into customer order behavior by focusing on shipped orders with even order IDs and filtering out specific regions based on pin codes. The results include the total quantity of products purchased by each customer, which can help the company analyze customer purchasing patterns and manage inventory more effectively.

ORDER_ID	CUSTOMER_ID	CUSTOMER_FULL_NAME	TOT_QTY
10008	7	MS. ASHWATHI BHATT	25
10022	23	MS. ANNA PINNOCK	2
10024	32	MR. HANS ZIMMER	2
10030	52	MS. SUCHIRITHAA EKANAYAKE	2
10032	7	MS. ASHWATHI BHATT	7
10034	19	MS. BHARTI SUBHASH	2
10036	24	MR. BRIAN GRAZER	4
10040	3	MS. KOMAL CHOUDHARY	2
10042	26	MR. STEPHEN E. RIVKIN	2
10046	3	MS. KOMAL CHOUDHARY	1
10048	33	MS. NISEEMA ZIMMER	1
10054	19	MS. BHARTI SUBHASH	3
10064	35	MR. THOMAS NEWMAN	3
10068	51	MR. AHMAD BIN GH AZALI	3
10070	10	MS. BIDHAN C.ROY	3

Frequent Buyers: MS. Ashwathi Bhatt appears multiple times with varying quantities (25 and 7), indicating a significant purchase pattern.

Diversity of Customers: The results include a range of customer names and genders, showing the diverse clientele of the company.

Order Analysis: By analyzing total quantities for each customer, the company can identify high-value customers and tailor marketing strategies accordingly.

This data is incredibly useful for managing inventory, nurturing customer relationships, and gaining insights into sales trends across various customer demographics.

Conclusion

In conclusion, our analysis of the orders database for Reliant Retail Limited has provided valuable insights that can drive the company's growth and enhance decision-making. By exploring customer demographics and purchasing behaviors, we can tailor marketing efforts and build stronger relationships with our customers. The inventory analysis has pinpointed products that need restocking or discounts, ensuring we meet customer demand effectively. Additionally, examining shipping performance, particularly with partners like DHL, has given us clarity on logistics and delivery efficiency. Overall, this project emphasizes the power of data-driven strategies in improving customer satisfaction and optimizing operations, setting Reliant Retail Limited up for success in the competitive online market.